

A Mini Project Report

on

NutriAi: Intelligent Personalized Nutrition Companion

Submitted in partial fulfillment of the requirements for the award of the degree of

Bachelor of Engineering

in

Computer Science and Engineering

By

Sanjay Samala 1608-22-733-097

Under the guidance of

Mrs. B. Meenakshi

Assistant Professor



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Matrusri Engineering College

Accredited by NBA & NAAC

(Affiliated to Osmania University, Approved by AICTE)

Saidabad, Hyderabad-500059 (2024-2025)

Department of Computer Science and Engineering

Matrusri Engineering College

Accredited by NBA & NAAC

(Affiliated to Osmania University, Approved by AICTE)

Saidabad, Hyderabad- 500059

(2024-2025)



CERTIFICATE

This is to Certify that A Mini Project report entitled “**NutriAi: Intelligent Personalized Nutrition Companion**” is being submitted by **Sanjay Samala (1608-22-733-097)** in partial fulfillment of the requirement of the award for the degree of Bachelor of Engineering in “Computer Science and Engineering” O.U., Hyderabad during the year 2024-2025 is a record of bonafide work carried out by them under my guidance. The results presented in this project have been verified and are found to be satisfactory.

Project Guide

Mrs. B. Meenakshi

Assistant Professor,

Dept of CSE

H.O.D

Dr. T. Raghunadha Reddy,

Associate Professor & HoD,

Dept. of CSE

External Examiner(s)



Department of Computer Science and Engineering

Matrusri Engineering College

Accredited by NBA & NAAC

(Affiliated to Osmania University, Approved by AICTE)

Saidabad, Hyderabad-500059 (2024-2025)

DECLARATION

I, **Sanjay Samala H.No 1608-22-733-097**, hereby certify that the minor project entitled “**NutriAi: Intelligent Personalized Nutrition Companion**” is submitted in the partial fulfilment of the required for the award of the degree of Bachelor of Engineering in Computer Science and Engineering. This is a record work carried out by us under the guidance of Mrs. B. Meenakshi, Assistant Professor , CSE, Matrusri Engineering College, Saidabad. The results embodied in this report have not been reproduced/copied from any source. The results embodied in this report have not been submitted to any other university or institute for the award of any other degree or diploma.

Sanjay Samala - 1608-22-733-097

ACKNOWLEDGEMENT

It is our privilege and pleasure to express my profound sense of respect, gratitude and indebtedness to our guide Mrs. B.Meenakshi, Assistant Professor, Department of Computer Science and Engineering, Matrusri Engineering College, for his inspiration, guidance, cogent discussion, constructive criticisms and encouragement throughout this dissertation work.

I express our sincere thanks to Project Coordinator Mrs. K. Shalini, Assistant Professor, Department of Computer Science and Engineering, Matrusri Engineering College, for her valuable suggestions and constant helping in completing the work.

I express our sincere gratitude to Dr. T. Raghunadha Reddy, Professor & Head of the Department of Computer Science and Engineering, Matrusri Engineering College, for his precious suggestions, motivation and co-operation.

I express our sincere thanks to Dr. G. Amarendar Rao, Principal, Matrusri Engineering College, Saidabad, Hyderabad, for his encouragement and constant help.

I extend our sincere thanks to all the teaching and non-teaching staff of Computer Science and Engineering Department for their support and encouragement.

Sanjay Samala - 1608-22-733-097

ABSTRACT

NutriAI: Intelligent Personalized Nutrition Companion is an AI-powered dietary guidance platform designed to deliver real-time, science-backed meal plans, recipe suggestions, and nutrition advice. Built with Streamlit for the interactive front-end and integrated with the Hugging Face API, NutriAI provides a secure, scalable, and user-friendly environment for individuals to manage their dietary needs effectively.

The platform features a conversational interface that allows users to input preferences, dietary restrictions, and health goals in natural language. Advanced AI models process these inputs to generate personalized recommendations, while multiple model options enable flexibility in response style and detail. Streamlit's secrets manager ensures secure API key management, maintaining privacy and data security.

NutriAI addresses the limitations of generic diet plans by adapting recommendations dynamically to user feedback and changing goals. Designed for scalability, it supports potential enhancements like voice-based interaction, image-based meal recognition, multilingual support, and wearable device integration.

By combining accessibility, personalization, and adaptability, NutriAI bridges the gap between professional nutrition consultations and everyday convenience. It empowers users to make informed dietary decisions, adopt healthier eating habits, and sustain long-term wellness through an engaging, interactive, and intelligent digital companion.

CONTENTS

1. INTRODUCTION.....	8
1.1 MOTIVATION.....	8
1.2 PROBLEM STATEMENT.....	8
1.3 PROJECT OBJECTIVE.....	8
2. LITERATURE SURVEY.....	9
2.1 EXISTING SYSTEM.....	10
2.2 LIMITATIONS OF EXISITNG SYSTEM.....	10
3. SOFTWARE REQUIREMENT SPECIFICATION.....	11
3.1 SOFTWARE REQUIREMENTS	11
3.2 HARDWARE REQUIREMENTS.....	11
3.3 FUNCTIONAL REQUIREMENTS.....	11
3.4 NON-FUNCTIONAL REQUIREMENTS.....	12
4. SYSTEM DESIGN.....	13
4.1 SYSTEM DESIGN.....	13
4.2 SYSTEM ARCHITECTURE.....	13
4.3 UML DESIGN.....	14
4.3.1 CLASS DIAGRAM.....	14
4.3.2 USECASE DIAGRAM.....	16
4.3.3 SEQUENCE DIAGRAM.....	16
4.3.4 ACTIVITY DIAGRAM.....	17
4.4 TECHNOLOGY DESCRIPTION.....	18
5. IMPLEMENTATION.....	19
5.1 IMPLEMENTATION PROCESS.....	19
5.2 MODULES.....	20
5.3 EXECUTABLE CODE.....	22

6. TESTING.....	26
6.1 TYPES OF TESTING.....	26
7. RESULTS.....	27
8. CONCLUSION.....	30
8.1 FUTURE SCOPE.....	30
9. REFERENCES.....	31
10. APPENDICES.....	32

1.INTRODUCTION

1.1 MOTIVATION

In today's fast-paced and health-conscious world, individuals are increasingly seeking personalized guidance for maintaining a balanced diet and healthy lifestyle. However, the vast amount of conflicting nutritional information available online often leads to confusion, poor dietary decisions, and unsustainable eating habits. Traditional methods of dietary planning—such as consulting nutritionists or following generic meal charts—can be time-consuming, costly, and lack adaptability to an individual's unique needs. This growing challenge motivates the need for an intelligent, always-accessible system that can provide reliable, science-backed, and personalized nutrition advice. NutriAI addresses this by combining the capabilities of advanced AI language models, real-time recipe generation, and user-friendly interaction to deliver tailored diet plans and healthy meal suggestions that adapt to user preferences, dietary restrictions, and health goals.

1.2 PROBLEM STATEMENT

Limitations of Existing System

- Generic meal plans that fail to consider individual health conditions, cultural preferences, and dietary restrictions.
- Inability to adapt recommendations dynamically based on user feedback or changing health goals.
- Lack of credibility verification for online nutritional information, leading to misinformation.
- Limited availability and high cost of professional nutritionists for personalized guidance.
- Fragmented resources requiring users to consult multiple platforms for recipes, nutrition facts, and diet tips.

1.3 PROJECT OBJECTIVE

The primary objective of this project is to design and implement an AI-powered nutrition assistant that:

- Generates personalized meal plans based on user preferences, dietary restrictions, and health goals.
- Provides real-time recipe suggestions using advanced AI language models.
- Understands and processes natural language queries for nutrition-related advice.
- Offers multiple AI model options to allow users flexibility in response style and detail.
- Integrates secure API key management using Streamlit's secrets manager.
- Maintains a simple, intuitive, and interactive chat-based interface for ease of use.
- Supports scalability for adding future features such as voice input, image-based meal recognition, and multilingual capabilities.

2. LITERATURE SURVEY

1. AI-Powered Dietary Recommendation Systems

Research in AI-driven nutrition platforms has shown that integrating natural language processing with dietary databases can significantly improve personalization in meal recommendations. These systems leverage machine learning to align diet suggestions with user preferences, allergies, and nutritional needs. However, most implementations are either restricted to static datasets or lack real-time conversational capability.

2. Recipe Generation using Large Language Models (LLMs)

LLMs such as LLaMA and GPT have demonstrated strong capabilities in generating contextually relevant recipes based on user-provided constraints. Studies highlight their flexibility in adapting to dietary restrictions, but also emphasize the importance of verifying nutritional accuracy.

3. Nutritional Information Verification and Trustworthiness

Online diet platforms often rely on unverified sources, leading to misinformation. Research suggests integrating credible nutritional databases like USDA FoodData Central to ensure scientific validity in AI-generated outputs.

4. Conversational User Interfaces in Health Applications

Conversational agents in healthcare and nutrition increase user engagement and adherence to recommendations. However, lack of personalization and follow-up tracking remains a limitation in existing systems.

5. Multi-Model AI Integration for Enhanced User Experience

Offering multiple AI models in a single platform allows users to select the response style and depth that suits them best. Studies show this approach enhances satisfaction but requires careful API management to maintain performance and cost efficiency.

Title	Contribution	Limitations
AI-Powered Dietary Recommendation Systems	Improves personalization and aligns diet with user needs	Often static, lacking real-time interaction
Recipe Generation using LLMs	Flexible recipe suggestions tailored to constraints	Nutritional accuracy may vary without database integration
Nutritional Information Verification	Increases trustworthiness of recommendations	Requires up-to-date, credible databases
Conversational UI in Health Apps	Enhances engagement and user adherence	Limited personalization and tracking
Multi-Model AI Integration	Offers variety in responses for better UX	Requires performance and cost balancing

Table 2.1: Literature Survey Overview

2.1 Existing System

Most existing nutrition recommendation systems rely on static, generic meal plans or predefined recipe collections. While some applications integrate basic filtering options for preferences and allergies, they often lack real-time adaptability and deeper personalization. Many platforms depend on fixed databases without incorporating evolving dietary trends, cultural variations, or user feedback loops. Furthermore, existing systems generally provide one-way information delivery, with minimal conversational engagement or interactive meal planning features.

2.2 Limitations of Existing System

- Generic meal plans that fail to consider individual health conditions, cultural preferences, and dietary restrictions.
- Inability to adapt recommendations dynamically based on user feedback or changing health goals.
- Lack of credibility verification for online nutritional information, leading to misinformation.
- Limited availability and high cost of professional nutritionists for personalized guidance.
- Absence of real-time, on-demand dietary advice and meal planning support.
- Fragmented resources requiring users to consult multiple platforms for recipes, nutrition facts, and diet tips.
- Minimal interactivity and engagement in most existing diet recommendation systems.

3. SOFTWARE REQUIREMENT SPECIFICATION

This chapter outlines the essential software and hardware components required to implement the NutriAI system, along with a detailed specification of its functional and non-functional requirements.

3.1 SOFTWARE REQUIREMENTS

- **Operating System:** Windows 10 / Linux (Ubuntu 20.04 or higher) / macOS Monterey or later
- **Programming Language:** Python 3.8+

Libraries & Frameworks:

- streamlit (for building the interactive web application)
- requests (for API communication with Hugging Face)
- pandas (for handling structured nutritional data if integrated)
- numpy (for data manipulation)
- Pillow (optional, for image handling in future meal recognition features)
- SpeechRecognition & gTTS (optional, for voice input/output in future scope)
- dotenv or Streamlit secrets (for secure API key storage)

3.2 HARDWARE REQUIREMENTS

- **System:** Intel i3 or above / AMD Ryzen 3 or above
- **RAM:** Minimum 4 GB (8 GB recommended for smooth model responses)
- **Storage:** At least 500 MB free space
- **Graphics:** Optional (if adding image recognition features in the future)
- **Internet:** Stable broadband connection for real-time API communication

3.3 FUNCTIONAL REQUIREMENTS

These define the core operations and user-facing capabilities of the system, which are directly observable in the final application:

- **Model Selection:**

Users can select from multiple AI models (e.g., LLaMA 2B and others) for varied response styles and accuracy.

- **Natural Language Query Processing:**

Users can ask nutrition-related questions in plain language for personalized advice.

- **Personalized Meal Plan Generation:**

AI suggests meal plans based on user preferences, health goals, and dietary restrictions.

- **Real-Time Recipe Suggestions:**

Provides step-by-step cooking instructions tailored to the user's query.

- **Secure API Key Management:**

Utilizes Streamlit's secret storage to protect Hugging Face API keys.

- **Interactive Chat Interface:**

Displays queries and AI responses in a conversational format.

- **Future-Ready Features (Optional):**

Voice-based interaction, multilingual support, and meal image analysis.

3.4 NON-FUNCTIONAL REQUIREMENTS

These define the overall quality and performance standards that the system must uphold:

- **Portability:**

Should function on Windows, Linux, and macOS platforms with minimal changes.

- **Security:**

Protects API keys and sensitive user data using secure storage methods.

- **Maintainability:**

Modular code design for easy updates, bug fixes, and feature enhancements.

- **Reliability:**

Consistent and accurate generation of meal plans and recipes without system crashes.

- **Scalability:**

Capable of supporting more AI models and additional features without major rework.

- **Performance:**

Query processing and response generation should occur within acceptable time limits (<3 seconds for typical queries).

- **Reusability:**

Core components such as query handling, API communication, and response formatting should be reusable in other health-related applications.

- **Flexibility:**

Should allow integration with third-party nutrition databases, wearable fitness trackers, or cloud-based storage in future versions.

4. SYSTEM DESIGN

4.1 SYSTEM DESIGN

In this phase, the architecture and module-level components of the NutriAI system are carefully designed based on the previously defined software requirement specifications. The design ensures that the system meets its usability, accuracy, and scalability goals.

4.1.1 High-Level Design (HLD)

Module Descriptions and Names:

- **User Interface Module:** Interactive Streamlit-based UI for chat interaction, model selection, and query input.
- **Query Processing Module:** Handles natural language inputs, formats them for API requests, and processes model responses.
- **AI Model Integration Module:** Connects to Hugging Face API, enabling the use of multiple AI models (e.g., LLaMA 2B and others) for generating diet plans and recipes.
- **Recommendation Engine Module:** Generates meal plans, recipes, and nutrition advice tailored to user preferences, restrictions, and goals.
- **Secure Configuration Module:** Manages API keys securely using Streamlit's secrets manager.
- **(Optional) Data Persistence Module:** Stores user preferences and past recommendations for improved personalization.

Functionality Outline:

- **User Interaction:** Users type nutrition or recipe-related queries into the chat interface.
- **Model Selection:** The selected AI model is loaded for response generation.
- **Query Processing:** User queries are pre-processed and sent to the Hugging Face API.
- **AI Response Generation:** The model returns structured meal plans, recipes, or tips.
- **Display of Output:** The UI presents results in a clean, conversational style.
- **Optional Feedback Loop:** User ratings or feedback improve future recommendations.

Interface Relationships and Dependencies:

- **UI Module** depends on Query Processing Module for sending user input.
- **Query Processing Module** depends on AI Model Integration Module for generating responses.
- **Recommendation Engine** relies on AI Model Integration for raw data and processes it into user-friendly output.
- **Secure Configuration Module** is required by AI Model Integration Module for authentication.

Database Tables and Key Elements (optional if personalization is added):

- **User_Profile:** Stores preferences, dietary restrictions, health goals.
- **Interaction_History:** Logs previous queries and responses.
- **Model_Performance:** Tracks average response time and accuracy feedback.

Architecture Diagrams and Technology Stack:

- **Frontend:** Streamlit web UI.
- **Backend Modules:** Python-based modules for query handling, AI model communication, and formatting.
- **APIs:** Hugging Face API for LLM inference.
- **System Type:** Web-based, cross-platform application.
- **Architecture Type:** Modular component-based design.

4.2 SYSTEM ARCHITECTURE:

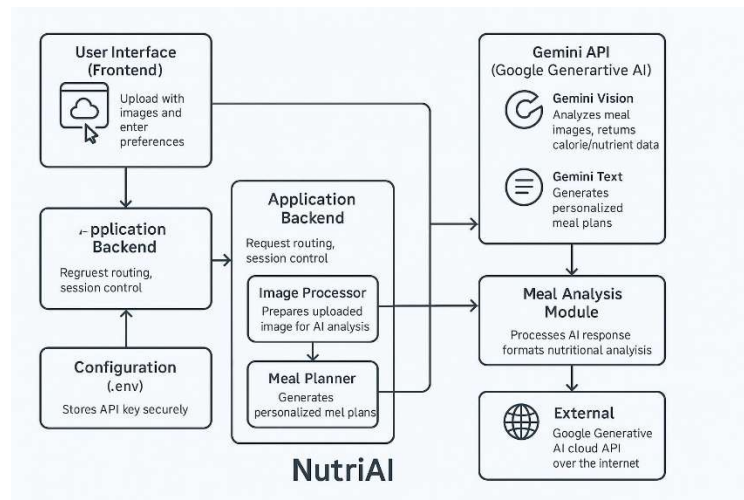


Fig.no.1. System Architecture

4.3 UML Design:

Unified Modeling Language (UML) is a general purpose modeling language. The main aim of UML is to define a standard way to visualize the way a system has been designed. It is quite similar to blueprints used in other fields of engineering.

UML is not a programming language; it is rather a visual language. We use UML diagrams to portray the behavior and structure of a system, UML helps software engineers, businessmen and system architects with modeling, design and analysis. The Object Management Group (OMG) adopted Unified Modeling Language as a standard in 1997.

It's been managed by OMG ever since. International Organization for Standardization (ISO) published UML as an approved standard in 2005. UML has been revised over the years and is reviewed periodically.

Do we really need UML?

- Complex applications need collaboration and planning from multiple teams and hence require a clear and concise way to communicate among them.
- Businessmen do not understand code. So UML becomes essential to communicate with non programmer's essential requirements, functionalities and processes of the system.
- A lot of time is saved down the line when teams are able to visualize processes, user interactions and static structure of the system.
- UML is linked with object oriented design and analysis. UML makes the use of elements and forms associations between them to form diagrams.

The Primary goals in the design of the UML are as follows:

- Provide users a ready-to-use, expressive visual modeling Language so that they can develop and exchange meaningful models.
- Provide extensibility and specialization mechanisms to extend the core concepts.
- Be independent of particular programming languages and development process.
- Provide a formal basis for understanding the modeling language.
- Encourage the growth of OO tools market.
- Support higher level development concepts such as collaborations, frameworks, patterns and components.- Integrate best practices.

Types of UML Diagrams:

- Structural Diagrams:

Capture static aspects or structure of a system. Structural Diagrams include: Component Diagrams, Object Diagrams, Class Diagrams and Deployment Diagrams.

- Behavior Diagrams:

Capture dynamic aspects or behavior of the system. Behavior diagrams include: Use CaseDiagrams, State Diagrams, Activity Diagrams and Interaction Diagrams.

4.3.1 CLASS DIAGRAM:

In software engineering, a class diagram in the Unified Modeling Language (UML) is a type of static structure diagram that describes the structure of a system by showing the system's classes, their attributes, operations (or methods), and the relationships among the classes. It explains which class contains information.

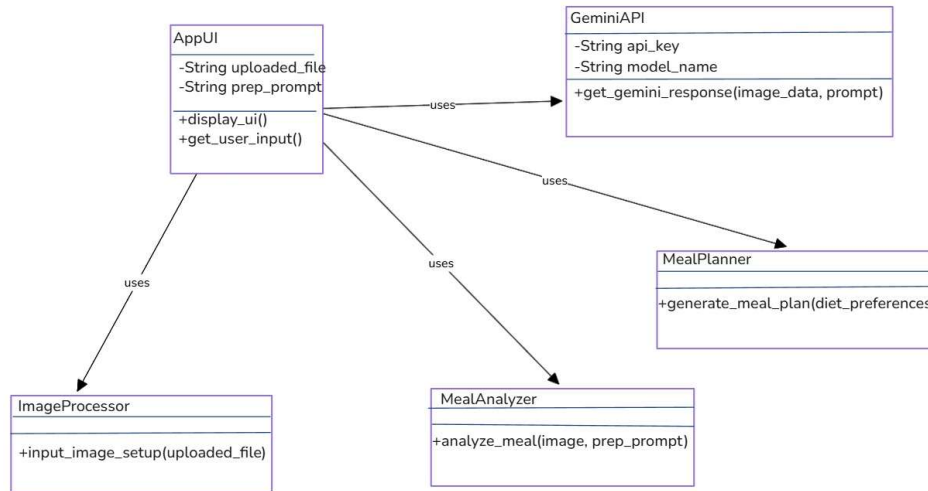


Fig.no.2. Class Diagram

4.3.2 USE CASE DIAGRAM:

A use case diagram in the Unified Modeling Language (UML) is a type of behavioral diagram defined by and created from a Use-case analysis. Its purpose is to present a graphical overview of the functionality provided by a system in terms of actors, their goals (represented as use cases), and any dependencies between those use cases. The main purpose of a use case diagram is to show what system functions are performed for which actor. Roles of the actors in the system can be depicted.

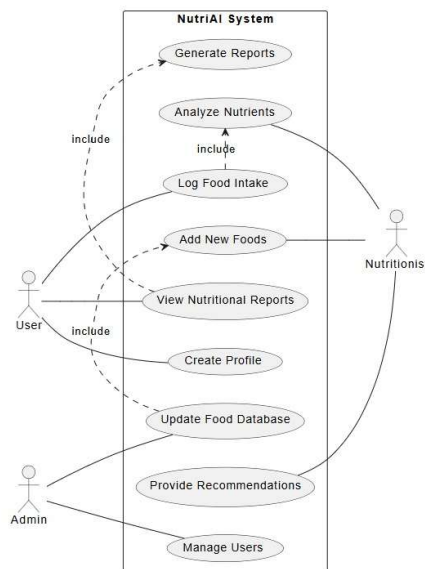


Fig.no.3. Use Case Diagram

4.3.3 SEQUENCE DIAGRAM:

A sequence diagram in Unified Modeling Language (UML) is a kind of interaction diagram that shows how processes operate with one another and in what order. It is a construct of a Message Sequence Chart. Sequence diagrams are sometimes called event diagrams, event scenarios, and timing diagrams.

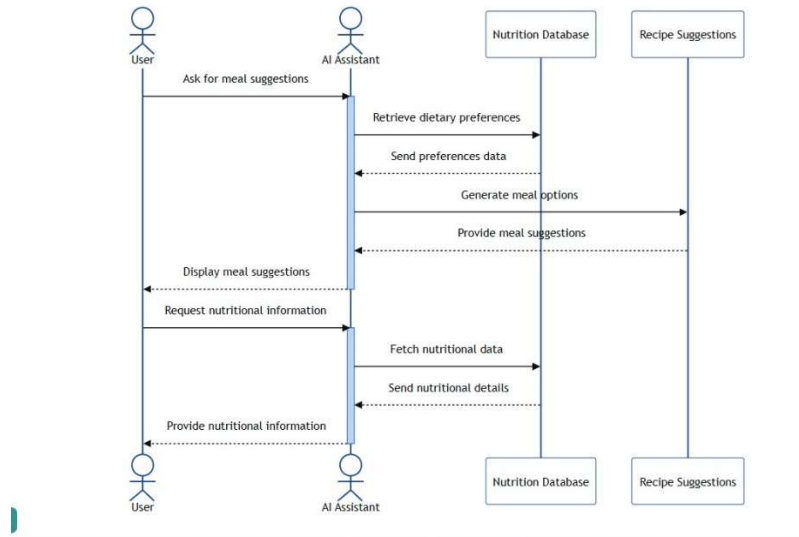


Fig.no.4. Sequence Diagram

4.3.4 ACTIVITY DIAGRAM:

In UML, an activity diagram is used to display the sequence of activities. Activity diagrams show the workflow from a start point to the finish point detailing the many decision paths that exist in the progression of events contained in the activity.

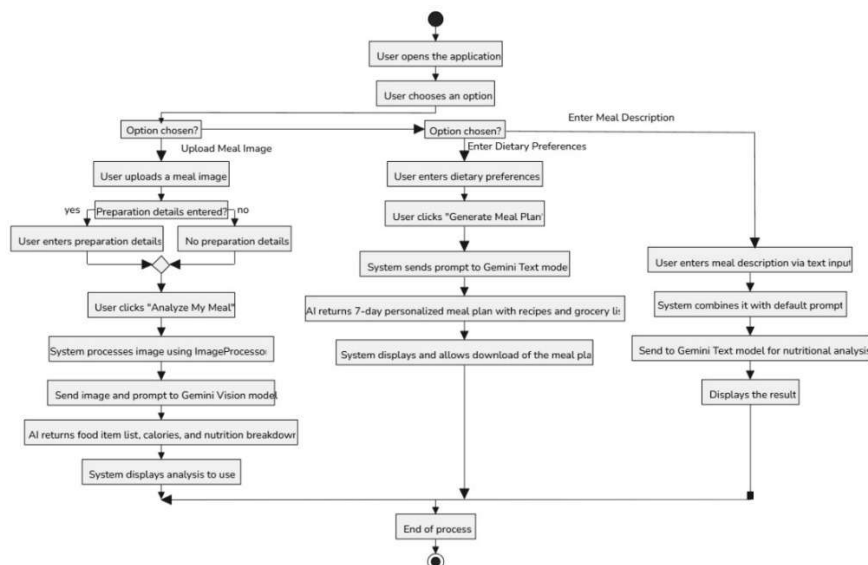


Fig.no.5. Activity Diagram

4.4 TECHNOLOGY DESCRIPTION

The development of the NutriAI platform integrates modern AI capabilities, secure backend handling, and an interactive front-end framework to deliver personalized nutrition guidance in real time. The system leverages natural language processing, API-based AI model integration, and a conversational user interface to ensure an engaging, accurate, and user-friendly experience.

NutriAI is built using Streamlit for its lightweight yet powerful web-based UI capabilities and the Hugging Face API for AI-driven meal plan generation and nutritional recommendations. The architecture ensures scalability, allowing future integration of features such as voice interaction, image-based meal recognition, and multilingual support. To maintain security and confidentiality, Streamlit's secrets manager is used for API key management and sensitive data handling.

Programming Language and Core Tools

- **Python 3.8+**

Python is a versatile, high-level programming language widely used in AI, data science, and web application development. It offers a rich ecosystem of libraries for natural language processing, API integration, and data handling, making it ideal for NutriAI's functionality.

- **Streamlit (Front-End Framework)**

A Python-based, open-source framework used to develop the interactive chat-based user interface. Streamlit allows rapid deployment of data-driven applications with minimal overhead, ensuring smooth user interaction and dynamic content rendering.

- **Hugging Face API**

Provides access to state-of-the-art AI models for generating meal plans, recipes, and nutritional advice based on user input.

- **Streamlit Secrets Manager**

Enables secure storage and retrieval of API keys and other sensitive credentials without exposing them in the application code.

5. IMPLEMENTATION

5.1 Implementation Process

The implementation of the NutriAI system follows a structured and modular development process to ensure seamless AI integration, user interactivity, and secure handling of sensitive data. The stages are carefully planned to deliver a fully functional, scalable, and user-friendly nutrition recommendation platform.

1. Environment Setup

- **Hardware and Software Installation:** Install Python 3.8 or higher, Streamlit, and required libraries (requests, streamlit, huggingface_hub, etc.) along with a suitable IDE (VS Code or PyCharm).
- **Library Configuration:** Setup a virtual environment and install dependencies via pip. Configure environment variables using Streamlit's secrets manager for secure API key storage.
- **Interface Preparation:** Initialize Streamlit for the web-based chat interface layout and navigation structure.

2. Development

Frontend (User Interface) Development

- **Chat Interface:** Develop an interactive chat-based UI in Streamlit for real-time user interaction.
- **Input Panels:** Create input fields for user preferences, dietary restrictions, and health goals.
- **Dynamic Response Display:** Implement real-time display of AI-generated meal plans and recipes.

Backend (Core Logic) Development

- **AI Model Integration:** Connect with Hugging Face API for meal plan and recipe generation.
- **Recommendation Engine:** Implement logic to tailor diet plans based on user inputs and preferences.
- **Secure Config Module:** Use Streamlit's secrets manager for API key handling.
- **Output Formatting:** Generate and display structured responses with meal details, ingredients, and preparation steps.

3. Testing

- **Unit Testing:** Validate API calls, data parsing, and UI components.
- **Integration Testing:** Ensure smooth data flow from user input → AI model → formatted output.
- **Edge Case Testing:** Test for incomplete inputs, unusual dietary restrictions, and invalid API responses.
- **User Acceptance Testing (UAT):** Collect feedback from sample users and refine UI/UX.

4. Deployment

- **Cloud Deployment:** Deploy on Streamlit Cloud or similar hosting platforms.
- **Cross-Platform Testing:** Validate performance across devices and browsers.
- **Distribution:** Share the deployed link for public or restricted access.

5. Training and Documentation

- **User Guide:** Create step-by-step instructions for using the NutriAI interface.
- **Technical Documentation:** Provide backend logic, API usage details, and architectural diagrams.

6. Maintenance and Updates

- **Bug Fixes:** Address UI, API, or logic issues promptly.
- **Feature Enhancements:** Add voice interaction, image recognition, or multilingual support based on feedback.
- **Security Updates:** Regularly review API handling and secure configuration practices.

5.2 MODULES

To ensure clarity, modularity, and maintainability, the NutriAI system is divided into the following core modules:

1. User Interface Module

Description: Provides the interactive chat-based interface through which users interact with NutriAI.

Key Features:

- Conversational chat window for user input and AI responses.
- Input fields for dietary preferences, restrictions, and health goals.
- Dynamic response display for meal plans, recipes, and nutritional advice.
- Cross-platform accessibility via browsers and Streamlit Cloud deployment.

2. User Input & Preference Handling Module

Description: Processes and validates user-provided data before sending it to the AI model.

Key Features:

- Captures preferences such as vegetarian, vegan, keto, or custom diets.
- Validates nutritional goals and formats input for API requests.
- Stores temporary session data for ongoing conversations.

3. AI Model Integration Module

Description: Connects with the Hugging Face API to retrieve AI-generated meal plans and recipes.

Key Features:

- Supports multiple AI model options for varied response styles.
- Sends structured prompts to models and parses returned data.
- Handles API response timeouts and errors gracefully.

4. Recommendation & Formatting Module

Description: Structures AI responses into user-friendly formats.

Key Features:

- Converts raw AI output into clear meal plan layouts.
- Highlights nutritional values, ingredients, and preparation steps.
- Provides adaptive suggestions based on previous user feedback.

5. Security & API Key Management Module

Description: Ensures secure handling of sensitive credentials.

Key Features:

- Utilizes Streamlit's secrets manager for API key storage.
- Prevents unauthorized access to API credentials.
- Enforces secure session handling and encrypted communication.

6. Future Enhancement Module (Planned)

Description: Placeholder for scalability features and advanced capabilities.

Key Features:

- Voice-based interaction for hands-free use.
- Image-based meal recognition and nutritional breakdown.
- Multilingual support and integration with wearable devices.

5.3 EXECUTABLE CODE

Project Directory Structure

1. Application – Streamlit-Based AI Nutrition Assistant

Path: E:\NutriAI\

app2.py : Main application file. Implements the NutriAI chat interface using Streamlit, connects to Hugging Face API for AI-driven meal plan and recipe generation, handles user preferences, and formats responses for display.

requirements.txt : Lists all Python dependencies required to run NutriAI (e.g., streamlit, requests, psycopg2, huggingface_hub).

2. Backend – PostgreSQL Database

Description:

NutriAI uses PostgreSQL as its backend database to store user session data, historical dietary preferences, and AI-generated recommendations for personalization and analytics.

Database Components:

Tables:

- **users** – Stores basic user profile and authentication details (if implemented).
- **preferences** – Stores dietary restrictions, health goals, and cultural food preferences.
- **recommendations** – Stores generated meal plans and recipes linked to user sessions.

Connection Handling:

- Managed via psycopg2 or SQLAlchemy in app2.py.
- Uses environment variables or Streamlit's secrets manager for secure database credentials.

3. Assets (Optional)

Currently, NutriAI does not require additional asset files. Images, icons, or branding materials can be added in future UI upgrades.

Core Backend Snippets (Python + Streamlit + PostgreSQL)

app2.py – Main Application & AI Interaction

```
import streamlit as st

import psycopg2

import requests

import json

# Load secrets securely

HF_API_KEY = st.secrets["hf_api_key"]

# PostgreSQL connection

def get_db_connection():

    return psycopg2.connect(

        host=st.secrets["db_host"],

        dbname=st.secrets["db_name"],

        user=st.secrets["db_user"],

        password=st.secrets["db_password"],

        port=st.secrets["db_port"]

    )

# Hugging Face API call

def generate_nutrition_plan(user_input, model_name="nutri-model"):

    headers = {"Authorization": f"Bearer {HF_API_KEY}"}

    payload = {"inputs": user_input}

    response = requests.post(f"https://api-inference.huggingface.co/models/{model_name}",

                             headers=headers, json=payload)

    return response.json()

# Save user query and AI response in PostgreSQL

def save_interaction(user_input, ai_response):

    conn = get_db_connection()

    cur = conn.cursor()

    cur.execute(

        "INSERT INTO recommendations (user_input, ai_response) VALUES (%s, %s)",
```

```

        (user_input, json.dumps(ai_response))
    )
    conn.commit()
    cur.close()
    conn.close()

# Streamlit UI
st.title("NutriAI – Your AI Nutrition Assistant")
user_query = st.text_input("Enter your health goal or meal preference:")
model_choice = st.selectbox("Choose AI Model", ["nutri-model", "diet-assistant", "healthy-eats"])
if st.button("Generate Plan"):
    if user_query:
        ai_output = generate_nutrition_plan(user_query, model_choice)
        st.write("### Your Personalized Nutrition Plan:")
        st.json(ai_output)
        save_interaction(user_query, ai_output)

```

Core Frontend Snippets (Streamlit Components)

Chat Interface – User Interaction

```

import streamlit as st

from app2 import generate_nutrition_plan

st.header("NutriAI Chat Interface")

user_message = st.text_area("Describe your dietary needs, restrictions, or goals:")

if st.button("Get Suggestions"):
    if user_message.strip():
        response = generate_nutrition_plan(user_message, "nutri-model")
        st.subheader("NutriAI Recommendations")
        for suggestion in response.get("meal_plan", []):
            st.write(f"🍽️ *{suggestion['meal']}* – {suggestion['description']}")

```


Model Selection & Preferences

```
import streamlit as st

st.subheader("AI Model & Preferences")

model_name = st.selectbox("Select AI Model", ["nutri-model", "diet-assistant", "healthy-eats"])

calorie_range = st.slider("Daily Calorie Target", 1200, 3500, (1800, 2500))

diet_type = st.radio("Diet Preference", ["Vegetarian", "Non-Vegetarian", "Vegan", "Keto"])

st.write(f"Model: {model_name}, Calorie Range: {calorie_range}, Diet: {diet_type}")
```

6. TESTING

The testing phase ensures that the NutriAI Intelligent Nutrition Assistant delivers accurate, reliable, and user-friendly dietary recommendations. Core components such as AI model interaction, recipe generation, PostgreSQL data storage, and user interface performance are tested for correctness, responsiveness, and security.

6.1 Testing Types

- **Unit Testing:**

Each backend function (Hugging Face API integration, PostgreSQL queries, preference handling) is tested independently to verify expected outputs for various inputs.

- **Integration Testing:**

Ensures smooth data flow between components — from user input → AI processing → recommendation generation → database storage → UI display.

- **System Testing:**

Validates the complete query-to-recommendation workflow in Streamlit, including model selection, user interaction, and database integration.

- **User Acceptance Testing (UAT):**

Conducted with sample users to verify ease of use, accuracy of meal plans, and overall satisfaction with recommendations and recipes.

- **Security Testing:**

Verifies secure handling of API keys (via Streamlit Secrets Manager) and protection of stored user data in PostgreSQL against unauthorized access.

- **Regression Testing:**

Ensures that new model integrations or UI improvements do not break existing features like chat responses, recipe generation, or saved recommendations.

- **Usability Testing:**

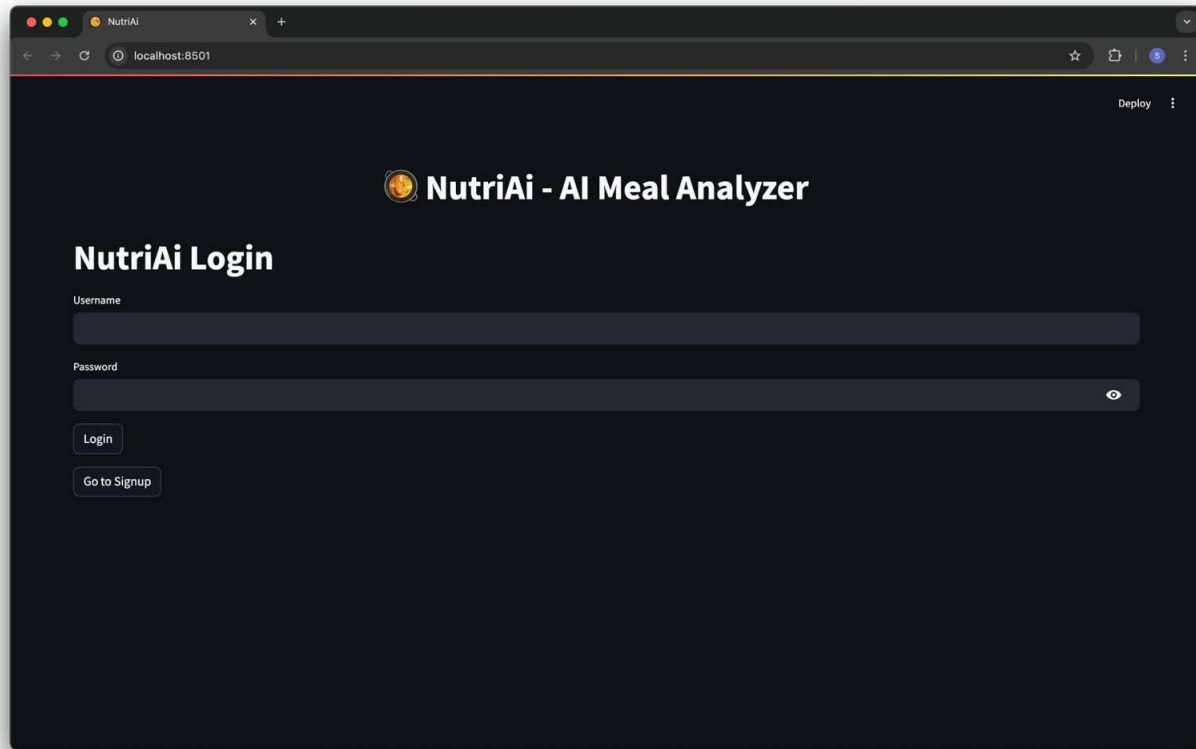
Tests the chat-based interface for clarity, navigation ease, and minimal user effort in generating personalized nutrition plans.

- **Compatibility Testing:**

Confirms proper functioning across desktop browsers (Chrome, Edge, Firefox) and various screen sizes, ensuring responsiveness in Streamlit UI.

7.RESULTS

1. Landing Page : (Login)



A screenshot of a web browser displaying the 'NutriAi - AI Meal Analyzer' login page. The page has a dark theme. At the top, there's a header with the site name and a 'Deploy' button. Below the header, the title 'NutriAi Login' is prominently displayed. The login form consists of two input fields: 'Username' and 'Password'. The 'Password' field has a toggle icon for visibility. Below the input fields are two buttons: 'Login' and 'Go to Signup'.

NutriAi - AI Meal Analyzer

NutriAi Login

Username

Password

Login

Go to Signup

2. SignUp Page :



A screenshot of a web browser displaying the 'NutriAi - AI Meal Analyzer' signup page. The page has a dark theme. At the top, there's a header with the site name and a 'Deploy' button. Below the header, the title 'NutriAi Signup' is prominently displayed. The signup form includes several input fields: 'Name', 'Username', 'Password' (with a toggle icon), 'Age' (with a numeric input and increment/decrement buttons), 'Gender' (a dropdown menu), 'Height (in cm)' (with a numeric input and increment/decrement buttons), 'Weight (in kg)' (with a numeric input and increment/decrement buttons), and 'Health Goal' (a dropdown menu). The 'Health Goal' dropdown is currently set to 'Lose Weight'.

NutriAi - AI Meal Analyzer

NutriAi Signup

Name

Username

Password

Age

25

Gender

Male

Height (in cm)

170

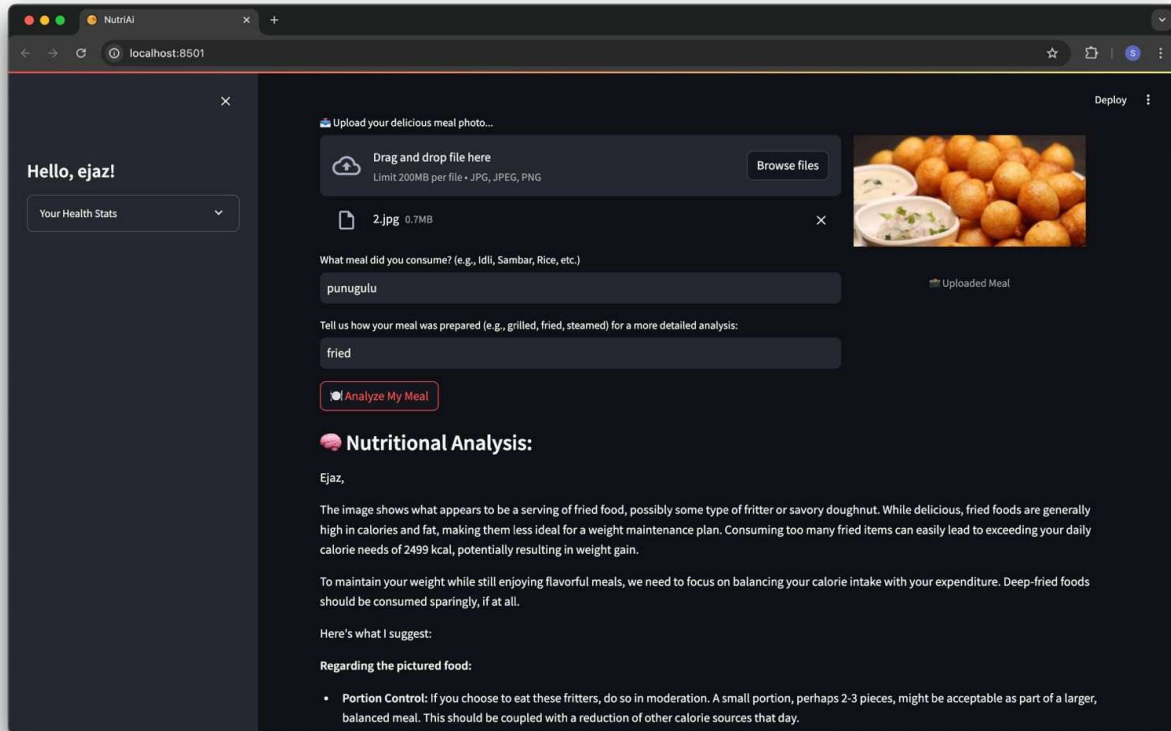
Weight (in kg)

70

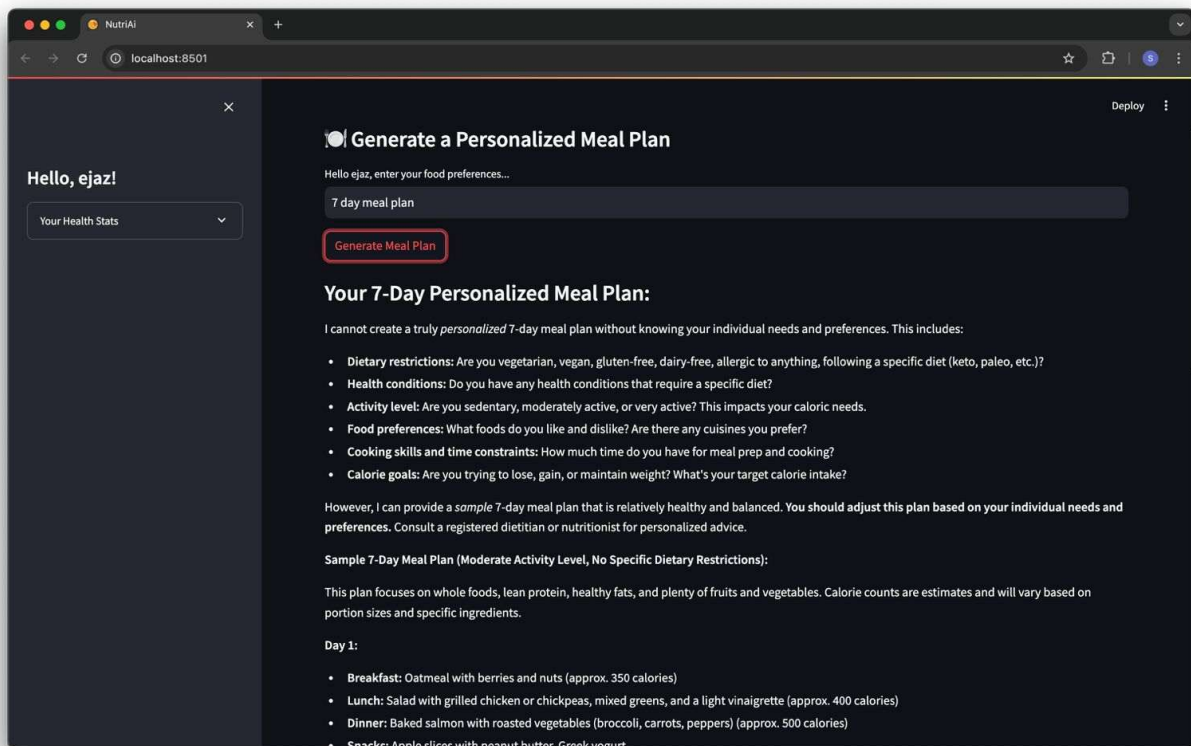
Health Goal

Lose Weight

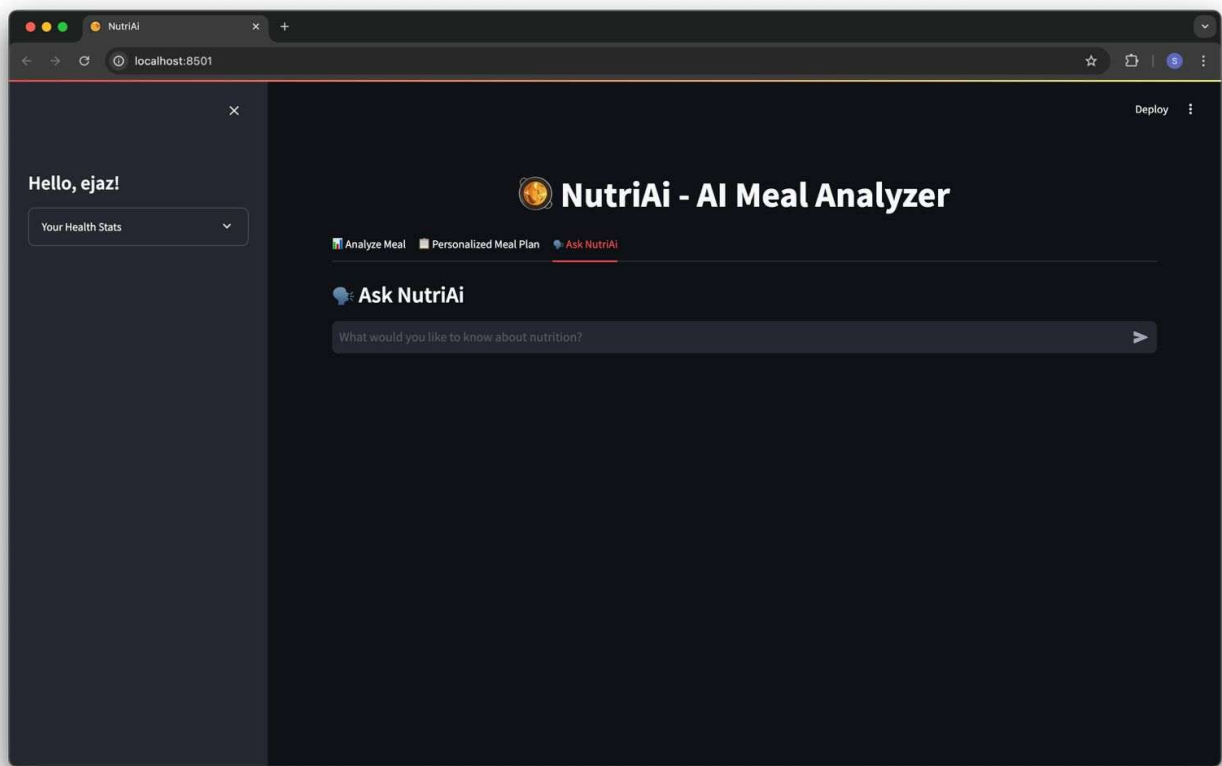
3. Meal Analyzer Page :



4. Meal Plan Page :



5. Nutrition Assistant :



8. CONCLUSION

The NutriAI Intelligent Nutrition Assistant successfully integrates advanced AI-powered language models, real-time recipe generation, and PostgreSQL-backed user data management into a unified, interactive, and personalized dietary recommendation platform. By allowing users to input their dietary preferences, health goals, and restrictions, the system generates customized, science-backed meal plans and recipes that adapt to individual needs.

The integration of Streamlit ensures a simple, accessible, and engaging chat-based interface, while secure API key management via Streamlit's Secrets Manager safeguards sensitive credentials. PostgreSQL storage enables persistence of user preferences and recommendations, allowing for data-driven personalization over time.

This project demonstrates the potential of combining AI-driven natural language processing with real-time nutritional intelligence to create a reliable, efficient, and user-friendly dietary planning tool. The system's modularity, scalability, and adaptability make it suitable for both academic research and real-world deployment in health and wellness applications. Overall, NutriAI represents a step forward in making personalized nutrition guidance accessible to anyone, anywhere, at any time.

8.1 FUTURE SCOPE

- **Voice Interaction Support:** Implement speech-to-text and text-to-speech capabilities for hands-free nutrition guidance.
- **Image-Based Meal Recognition:** Enable users to upload meal photos for AI-based nutritional breakdown and suggestions.
- **Multilingual Support:** Expand accessibility by supporting multiple languages in the chat interface and recipe outputs.
- **Integration with Fitness Trackers:** Sync with devices like Fitbit or Apple Health to tailor meal plans based on real-time activity data.
- **Offline Recommendations:** Implement a lightweight AI model for offline dietary advice without constant internet dependency.
- **Advanced Personalization:** Use machine learning to refine recommendations based on historical preferences and feedback.

9. REFERENCES

- **Ahmed, M. Shaheer et al. (2023).** AI-Driven Personalized Meal Recommendations Using Machine Learning. Discusses adaptive diet planning algorithms leveraging real-time user feedback. ResearchGate.
- **Stanford University (2024).** Advances in Nutritional AI Systems. Covers modern NLP approaches in dietary advice generation. Stanford AI Lab Publications.
- **World Health Organization (2024).** Healthy Diet Factsheet. Provides globally recognized nutritional guidelines for balanced diets. WHO.org.
- **Kumar, R. & Singh, P. (2023).** Streamlit for Interactive Health Applications. Demonstrates practical implementation of AI chatbots for wellness platforms. IEEE Xplore.
- **PostgreSQL Global Development Group (2024).** PostgreSQL Documentation – Latest Release. Official documentation for PostgreSQL database features and integration best practices. PostgreSQL.org.
- **OpenAI (2025).** Best Practices for AI Model Integration. Guidelines for securely and efficiently using AI APIs in production. OpenAI Technical Notes.
- **Harvard School of Public Health (2024).** Nutrition Source: Healthy Eating Plate. Evidence-based dietary recommendations. Harvard.edu.
- **Singh, A. et al. (2024).** Food Image Recognition for Calorie Estimation. Research on AI-based nutritional analysis using image processing. ACM Digital Library.

10. APPENDICES

Appendix A: Project Repository

Project Title: NutriAI – AI-Powered Personalized Nutrition Assistant

GitHub Repository Link: [GitHub Link]

This repository contains the complete source code for the NutriAI system, including:

- **Frontend:** Streamlit-based interactive chatbot interface for meal planning and recipe suggestions.
- **Backend:** Python-based API endpoints handling AI model integration, PostgreSQL queries, and recipe generation.
- **Database Schema:** PostgreSQL table structures for storing user profiles, preferences, and meal histories.
- **AI Integration Modules:** Functions for querying multiple AI models (e.g., LLaMA 2B and others) via API.
- **Secrets Management:** Streamlit's secrets.toml configuration for secure API key handling.
- **Sample Data:** Example nutritional datasets and recipe templates for testing and development.
- **Deployment Scripts:** Instructions and files for deploying the app locally or on cloud platforms like Streamlit Cloud or Heroku.